รายงานทดสอบความจุระบบลงเวลาด้วยใบหน้า (Face Kiosk) — สรุปการวัด สาเหตุ ข้อแนะนำ และการประมาณสเป็ค

ช่วงเวลาทดสอบ: 22–24 กันยายน 2026

เซิร์ฟเวอร์: `10.50.0.117` (production ภายใน) · URL `http://10.50.0.117:8880`

สเป็คเครื่อง: CPU 32 core · RAM 436 GB · GPU 2× NVIDIA Quadro RTX 5000 16 GB

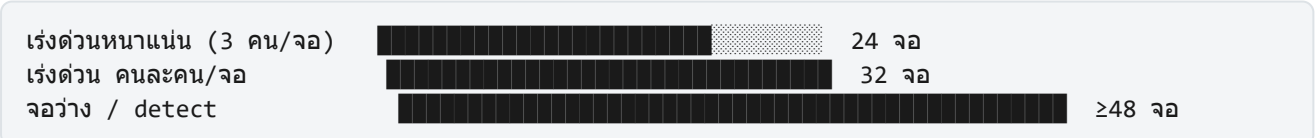
โค้ดล่าสุดที่วัด: `9d407e2d` (perf รอบ 2) + overlay quad-API (`a00e354c`)

เอกสารเทคนิคประกอบ: KIOSK-LOADTEST-SATURATION.md · PERF2-TABLE · PERF3-TABLE

---

1. สรุปสำหรับผู้บริหาร (อ่านหน้าเดียวจบ)

คำถาม	คำตอบจากผลวัดล่าสุด (รอบ 3)
จอคีออสที่ลื่นในชั่วโมงเร่งด่วน (หลายคนหน้ากล้อง)	**24 จอพร้อมกัน** (แนะนำวางแผนที่ 16–20)
ถ้าคนละคนต่อจอ / เปิดบนมือถือของตัวเอง	**32 คนพร้อมกัน** (แนะนำวางแผนที่ 24–28)
จอว่างรอคน (detect อย่างเดียว)	**≥ 48 จอ** ยังไม่ชนเพดาน
เทียบจุดเริ่มต้น (22 ก.ย.)	เพดานเร่งด่วน **5 → 24 จอ (~4.8 เท่า)** โดยไม่ซื้อฮาร์ดแวร์
ทรัพยากรที่ยังเหลือ	GPU ใช้เฉลี่ยเพียง **~16%** ต่อใบ · CPU ใช้ ~480% จาก 3,200%
รองรับพนักงาน 10,000 คนลงเวลาช่วงเดียวกันไหม	**ไม่ได้บนเครื่องเดียว** — ต้องสเกลหลายเครื่อง (ดู §7)



---

2. ไทม์ไลน์การทดสอบและสิ่งที่ปรับในแต่ละรอบ

วัน–เวลา (UTC+7)	รอบ	สภาพระบบ	ผลเพดาน rush×3 / rush×1 / idle
22 ก.ย. 11:25–12:30	Baseline (laptop RTX 5050)	process เดียว, worker 1	สร้างเครื่องมือ loadtest, ยืนยันวิธีวัด

23 ก.ย. 13:55–15:15	Server ครั้งแรก	`FACE_DEVICES=cuda:0,cuda:1` ใน process เดียว, worker 2, เฟรมทดสอบใหญ่ (~546 KB)	**5 / 8 / 12 จอ**
23 ก.ย. 16:00–17:40	Retest workers=2	เท่าเดิม + วัด GPU util จริง	พบ GPU0 util 16%, GPU1 ~2% → pin ผิด
23 ก.ย. 22:00–22:35	Dual-API (Phase 1–2)	แยก `dpu-api` (GPU0) + `dpu-api-gpu1` (GPU1), nginx `least_conn`, เฟรมสมจริง 78 KB, decode pool	**6 / 12 / 24 จอ**
23 ก.ย. 23:50–24:10	Perf รอบ 2	det@320 แล้ว embed ทันที, identify/check-in ใน memory (ไม่เขียน temp file), worker 3/process, หน้าบ้าน motion gating	**16 / 24 / ≥32 จอ**
24 ก.ย. 00:20–00:45	Perf รอบ 3	2 process ต่อ GPU (4 process), whisper_server ย้ายไป GPU1	**24 / 32 / ≥48 จอ**

นิยามโปรไฟล์ทดสอบ (จำลองพฤติกรรมคีออสจริงจาก `enroll-face.component.ts`):

- **idle** — จอมีคนอยู่ในเฟรมแต่ไม่ลงเวลา: ส่ง detect ทุก 90 ms ต่อเนื่อง (~11 เฟรม/วินาที/จอ)
- **rush × 3** — 3 คนหน้ากล้องพร้อมกัน: detect + identify (≤ 2 ค้าง) + check-in (≤ 3 ค้าง) ตลอด 30 วินาที
- **rush × 1** — คนเดียวต่อจอ (เช่น เปิดบนมือถือตัวเอง) ทำ identify + check-in ต่อเนื่อง

เกณฑ์ผ่าน (SLO): **detect p95 ≤ 550 ms · ทิ้งเฟรม ≤ 5% · session p95 ≤ 500 ms** — ตัวเลข "ใช้ได้" ในรายงานคือจำนวนจอสูงสุดที่ยังผ่านเกณฑ์นี้

> ตัวเลขทั้งหมดวัดด้วยโหลดที่ **หนักกว่าจริง**: loadgen ส่ง detect เต็มอัตราตลอด ไม่มี cooldown 5 นาทีหลังลงเวลา และไม่มี motion gating ของหน้าบ้าน — ของจริงจึงรับได้มากกว่านี้

---

3. ค่าเฉลี่ย / p95 ที่วัดได้ (รอบล่าสุด)

3.1 Latency ต่อจำนวนจอ — รอบ 3 (4 process, worker 3)

โปรไฟล์	จอ	detect p50	detect p95	ทิ้งเฟรม	identify p95	check-in p95	ผล
idle ×3	16	56 ms	72 ms	0.4%	—	—	OK
idle ×3	32	68 ms	124 ms	0.1%	—	—	OK
idle ×3	48	97 ms	173 ms	0.0%	—	—	OK
rush ×3	8	67 ms	120 ms	0.3%	223 ms	158 ms	OK
rush ×3	16	225 ms	383 ms	0.0%	452 ms	384 ms	OK
rush ×3	**24**	311 ms	**514 ms**	0.0%	614 ms	548 ms	**OK (ขอบ)**

rush ×3	32	483 ms	740 ms	0.0%	806 ms	779 ms	เกิน SLO
rush ×1	16	107 ms	179 ms	0.0%	244 ms	212 ms	OK
rush ×1	24	176 ms	307 ms	0.0%	336 ms	333 ms	OK
rush ×1	**32**	251 ms	**429 ms**	0.0%	465 ms	471 ms	**OK**
rush ×1	48	523 ms	817 ms	0.0%	730 ms	844 ms	เกิน SLO

3.2 การใช้ทรัพยากรเครื่องระหว่างโหลด (เฉลี่ยช่วงที่มีงาน)

รอบ	process API	GPU0 SM	GPU1 SM	CPU ต่อ process	CPU รวม API	ข้อสรุป
Retest 23 ก.ย.	1	~16% (peak 87%)	~2%	—	—	GPU1 ไม่ถูกใช้จริง (pin ผิด)
รอบ 2	2	~11% (peak 31%)	~11% (peak 35%)	~150% (p95 250%)	~300%	คอขวด CPU/GIL ต่อ process
รอบ 3	4	~18% (peak 36%)	~15% (peak 39%)	~120% (p95 200%)	~480%	GPU ยังเหลือ >80%; latency ไล่ตาม check-in

3.3 เวลาต่อคำขอที่โหลดต่ำ (4 จอ) — ใช้เป็น "ค่าฐาน" ของระบบ

งาน	ก่อนรอบ 2	หลังรอบ 2	หมายเหตุ
detect (1 เฟรม)	p50 49 ms · p95 181 ms	p50 55 ms · p95 90 ms	det@320 บน GPU + decode บน CPU pool
identify (1 หน้า)	p95 326 ms	**p95 187 ms**	ตัด detect 640 + ตัด disk I/O
check-in	p95 110 ms	p95 135 ms	จำกัดที่ DB (dedupe 5 นาที + insert)
GPU compute จริงต่องาน	~5–15 ms	~5–15 ms	ไม่ใช่คอขวด

---

4. สาเหตุที่ระบบช้า (จากหลักฐานที่วัด ไม่ใช่การเดา)

ลำดับ	คอขวด	หลักฐาน	สิ่งที่แก้แล้ว
1	**CPU/GIL ใน Python process เดียว** — decode JPEG, base64, JSON, WebSocket I/O แย่ง GIL กับ ONNX	process เดียว: GPU ว่าง 80%+ แต่ worker "ยุ่ง" ตลอด; แยก 2 process แล้วเพดาน idle 12 → 24 จอ; 4 process → ≥48	Dual API (รอบ dual) → Quad API (รอบ 3)
2	**GPU1 ไม่ถูกใช้จริง** — insightface ไม่ส่ง `device_id` ให้ CUDA	`nvidia-smi --query-compute-apps` เห็น python	ส่ง `provider_options` `device_id` +

	provider ทำให้ทั้ง `cuda:0` และ `cuda:1` ลง GPU0	เฉพาะ GPU0	`CUDA_VISIBLE_DEVICES` ต่อ process
3	**หน้าบ้านส่งเฟรมถี่เกินจำเป็น** — 11 เฟรม/วินาที/จอ แม้ภาพนิ่ง	idle 24 จอ = ~264 detect/วินาที คือโหลดหลัก	thumbnail diff + ปรับ interval ตามสถานะ track (รอบ 2)
4	**path identify/check-in เขียนไฟล์ + detect ซ้ำที่ 640** — เขียน temp → อ่านซ้ำ → detect 640 → embed → copy รูป	identify p95 300–700 ms ใน rush แม้ detect ยังดี	ทำใน memory + det@320 แล้ว embed จาก keypoints (รอบ 2)
5	**เฟรมทดสอบใหญ่เกินจริง** (~546 KB / 3,936 px) ทำให้ตัวเลขแย่กว่าจริง	decode 3.7 MP กิน CPU 15–25 ms/เฟรม	loadgen ใช้เฟรม 1280 px q70 (~78 KB) เท่าคืออสจริง
6	**GPU0 ถูกแชร์กับ whisper_server** (~4 GB)	compute-apps บน GPU0	ย้ายไป GPU1 (รอบ 3) — ผลต่อ latency น้อย เพราะ GPU ไม่ใช่คอขวด
7	**DB / dedupe ต่อ check-in** (ยังเหลืออยู่)	check-in p95 ไล่ตาม detect p95 ในทุก step ของรอบ 3; rush×3 ที่ 24 จอ check-in 548 ms	ยังไม่ได้แก้ — เป็นเพดานถัดไป

สิ่งที่ไม่ใช่คอขวด: GPU compute (RetinaFace 320 / ArcFace ~5–15 ms/งาน), nginx, uvicorn WS handshake (session p95 < 130 ms ทุก step), pgvector search เอง (HNSW `ef_search=40`)

---

5. ข้อแนะนำที่จำเป็น

5.1 การติดตั้งใช้งานจริงบนเซิร์ฟเวอร์นี้

1. **วางแผนติดตั้งไม่เกิน 16–20 จอ** ในโซนที่ชั่วโมงเร่งด่วนมีหลายคนหน้ากล้อง (เพดานเทส 24)
2. โซนที่คนละคนต่อจอ / มือถือส่วนตัว: **ไม่เกิน 24–28 คนพร้อมกัน** (เพดานเทส 32)
3. ใช้ overlay **quad-API** ตาม `docs/DEPLOY-SERVER.md` (`docker-compose.gpu.yml` + `docker-compose.gpu-x2.yml`) และคง `FACE_GPU_WORKERS=3`
4. เก็บเกณฑ์ UX: กรอบตอบสนอง ≤ 0.55 วินาที · ทิ้งเฟรม ≤ 5% — ถ้าเกินให้ลดจำนวนจอหรือขยายตาม §7
5. `whisper_server` ไม่อยู่ใน systemd — หลังรีบูตต้องรันใหม่พร้อม `CUDA_VISIBLE_DEVICES=1` (บันทึกใน DEPLOY-SERVER)

5.2 การปรับปรุงลำดับถัดไป (เรียงตามผลตอบแทน)

ลำดับ	รายการ	คาดผล	ความยาก
1	**ลดงาน DB ต่อ check-in** — cache dedupe ในmemory ต่อ process (TTL 5 นาที) ก่อน query, เขียน timestamp แบบ async batch	check-in p95 ลด 30–50% ใน rush	ต่ำ–กลาง
2	**3 process ต่อ GPU** (6 process) บนเครื่อง 32 core	rush×3 24 → ~32 จอ (ผลตอบแทนเริ่มลด)	ต่ำ (compose overlay)

3	**วัดด้วยคีออสจริง** (motion gating ทำงาน) แทน loadgen เต็มอัตรา	ได้ตัวเลขจริงที่สูงกว่าตาราง	ต่ำ
4	**แยก DB ออกจากเครื่อง API** เมื่อเกิน ~40 จอ	ตัด contention CPU/IO ระหว่าง API กับ Postgres	กลาง
5	**Face Core หลายเครื่อง + Redis queue**	สเกลเป็นร้อยจอ / หลายบริษัท	สูง — ทำเมื่อต้องข้ามเครื่องเท่านั้น

5.3 สิ่งที่ไม่ควรทำ

- ไม่ต้องซื้อ GPU เพิ่มเพื่อเครื่องนี้ — GPU ใช้เพียง ~16%
- ไม่ต้องเพิ่ม `FACE_GPU_WORKERS` เกิน 3 — A/B 2 vs 3 ไม่ต่างในระดับ noise; เกินนี้จะแย่ง GPU กันเอง
- ไม่ต้องแยก Face Core เพื่อ "ใช้ GPU ให้เต็ม" — ปัญหาอยู่ที่ CPU/DB ไม่ใช่ GPU

---

6. การประมาณสเป็ค — เครื่องเดียว

หลักคิด: **1 API process ≈ 1.2–1.5 CPU core เมื่อเต็มโหลด** และรับ rush ได้ประมาณ **6–8 จอ (3 คน/จอ)** หรือ **8–10 คน (คนละคน/จอ)**; GPU 1 ใบรองรับได้ 2–3 process โดยยังใช้ SM ไม่ถึง 40%

เป้าหมายพร้อมกัน (rush)	API process	CPU core แนะนำ	RAM	GPU	หมายเหตุ
≤ 6 จอ / ≤ 12 คน	1	4–8	16 GB	1 ใบ ระดับ RTX 4060 / T4	เครื่องเล็ก / สาขา
≤ 16 จอ / ≤ 24 คน	2 (1/GPU)	8–16	32 GB	1–2 ใบ	ค่าที่วัดได้รอบ 2
≤ 24 จอ / ≤ 32 คน	**4 (2/GPU)**	**16–32**	32–64 GB	**2 ใบ RTX 5000**	**เครื่องปัจจุบัน (รอบ 3)**
≤ 32 จอ / ≤ 48 คน	6 (3/GPU) + DB แยก	32	64 GB	2 ใบ	ประมาณการ — ยังไม่ได้วัด
> 48 จอ / > 64 คน	หลายเครื่อง	—	—	—	ดู §7

การ์ดจอ: งานเป็น RetinaFace 320 + ArcFace r50 ที่ ~5–15 ms/งาน — การ์ดระดับ 8–16 GB ทั่วไปเพียงพอ; **จำนวน CPU core และ process สำคัญกว่า GPU รุ่นแรง**

---

7. การประมาณสเป็ค — กรณี 10,000 คนลงเวลาช่วงเดียวกัน

7.1 แปลงเป็นอัตราคำขอ

สมมติฐาน	ค่า
พนักงาน	10,000 คน

หน้าต่างเวลาลงเวลา	20 นาที (07:50–08:10)
อัตราเฉลี่ย	≈ **8.3 คน/วินาที**
อัตราพีค (นาทีที่แน่นสุด ~3–5× ของเฉลี่ย)	≈ **25–45 คน/วินาที**
งานต่อ 1 คน (คืออสจริงหลัง motion gating)	detect ~15–25 เฟรม + identify 1–2 + check-in 1 ≈ 20–30 คำขอ
คำขอรวมช่วงพีค	≈ **600–1,200 คำขอ/วินาที**

7.2 เครื่องปัจจุบันรับได้เท่าไร

จากรอบ 3: 24 จอ rush×3 = ~72 คนหน้ากล้องพร้อมกัน ≈ **50–60 detect/วินาที + ~35 identify/วินาที + ~95 check-in/วินาที** โดยยังผ่าน SLO → เทียบเป็น **"ลงเวลาสำเร็จ" ประมาณ 3–6 คน/วินาที** ต่อเครื่อง (ขึ้นกับเวลาที่แต่ละคนยืนหน้ากล้อง ~4–8 วินาที)

ดังนั้น 1 เครื่อง ≈ 4,000–7,000 คนต่อ 20 นาที ถ้าโหลดกระจายสม่ำเสมอ — แต่ที่พีค 25–45 คน/วินาที จะล้นทันที

7.3 สเป็คที่แนะนำสำหรับ 10,000 คน / 20 นาที

องค์ประกอบ	จำนวน / สเป็ค	เหตุผล
จอ/จุดลงเวลา	**≥ 120–150 จุด** (คนละคนต่อจอ) หรือ ≥ 60 จอแบบหลายคน	45 คน/วินาที × 4–6 วินาที/คน ≈ 180–270 คนยืนพร้อมกันตอนพีค — จำนวนจอเป็นข้อจำกัดทางกายภาพก่อนซอฟต์แวร์
API/Face node	**4–6 เครื่อง** ระดับเครื่องปัจจุบัน (16–32 core, 2 GPU 16 GB) หรือ 8–12 เครื่องระดับ 1 GPU	แต่ละเครื่องรับ ~30 คนพร้อมกัน / ~5 ลงเวลาต่อวินาที; ต้องการ ~250 คนพร้อมกันตอนพีค + margin 30%
Load balancer	nginx / HAProxy `least_conn` แยกเครื่อง, sticky ต่อ WebSocket	กระจาย WS ต่อ connection
Database	PostgreSQL + pgvector แยกเครื่อง: 16 core, 64 GB, NVMe; connection pool รวม ≥ 300; พิจารณา read replica สำหรับ 1:N search	check-in insert ~50–100/วินาที + dedupe query + vector search 10,000 embedding (×5 มุม)
Object storage	NFS/S3 สำหรับรูป punch (~50 KB × 10,000 = ~500 MB/วัน)	หลาย node ต้องเขียนรูปที่เดียว
Queue (ทางเลือก)	Face Core + Redis เมื่อต้องการ fairness หลายบริษัท	ไม่จำเป็นถ้าแยก node ตามสาขา/บริษัทอยู่แล้ว

7.4 ทางเลือกลดสเป็ค (ทำได้ทันที ไม่ต้องซื้อ)

- **กระจายเวลา:** แบ่งกะ/แผนกลงเวลาต่างช่วง 10–15 นาที ลดพีคจาก 45 → ~15 คน/วินาที → ลดจำนวนเครื่องลงครึ่งหนึ่ง
- **แยกตามสาขา:** สาขาละเครื่อง (แต่ละสาขาไม่น่ามี > 1,000 คนลงเวลาพร้อมกัน) แล้ว sync ไปฐานกลาง
- **ลดงานต่อคน:** motion gating (มีแล้ว) + cooldown 5 นาที (มีแล้ว) + ทำ dedupe cache ใน memory (ข้อแนะนำ §5.2-1)

คำตอบตรง ๆ: 10,000 คนในช่วงเดียวกัน **ทำได้** แต่ต้องเป็นระบบ **4–6 เครื่องระดับปัจจุบัน + DB แยก + จุดลงเวลา ≥ 120** — ไม่ใช่เครื่องเดียว

---

8. วิธีทดสอบซ้ำ (สำหรับทีมเทคนิค)

```
# บนเซิร์ฟเวอร์ (quad API)
docker compose -f docker-compose.yml -f docker-compose.gpu.yml -f docker-compose.gpu-
x2.yml up -d --build

# จากเครื่องทดสอบ
export KIOSK_LOAD_BASE_URL=http://10.50.0.117:8880
python scripts/kiosk-load/kiosk_load.py --prepare --frame-width 1280 --jpeg-quality 70 \
  --profile rush --faces-per-kiosk 3 --kiosks 8,16,24,32 --hold 30 --jsonl results-
rush3.jsonl
python scripts/kiosk-load/summarize_capacity.py --idle ... --rush3 ... --rush1 ... --
workers 3 --out docs/TABLE.md
```

ไฟล์ดิบทุกรอบอยู่ใน `scripts/kiosk-load/.cache/` (ไฟล์ `results-<รอบ>-<โปรไฟล์>.jsonl` และ `hostmetrics-<รอบ>.csv`)

จัดทำจากผลวัดจริงบนเซิร์ฟเวอร์ 22–24 กันยายน 2026 · ตัวเลข "ประมาณการ" ใน §6–7 ยังไม่ได้วัดจริงและควรยืนยันด้วย loadtest ก่อนจัดซื้อ